

RigL on ODT

The Complete Implementation, Line by Line

Every RigL code fragment from the 39-chapter ODT reference, consolidated into build order, with each line explained

Extracted from	ODT_Complete_Reference_v2.pdf (142 pages, 39 chapters)
Source sections	41 RigL sections: 2.1-2.4, the 'Missing RigL Code' section of every chapter, and Chapter 39 in full
Components	7 required + 2 optional (AdamW, Conv1d), in dependency order
Target	STM32 Nucleo-F746ZG, Cortex-M7 at 216 MHz, 320 KB SRAM
Code	C99, every line numbered and explained individually
Also included	An engineering review of the defects found while extracting, and ready-to-compile RigL.h / RigL.c

What This Document Is

In the source reference, the RigL implementation is **not in one place**. It is distributed across 41 sections of 39 chapters: each chapter documents one source file of the ODT library and ends with a 'Missing RigL Code' section describing what that particular file still needs. Chapter 39 then gathers seven of those fragments, but not all of them, and in places states the code differently from the chapter it came from.

This document does the consolidation properly. It collects **every** RigL fragment, orders them by build dependency rather than by file name, explains every line of every fragment individually, and records where each piece came from so you can always go back to the original.

How to read a component chapter

- **Provenance bar** - the grey strip naming the exact section of the source PDF the code came from.
- **Purpose** - what the component does and which other components depend on it.
- **Numbered listing** - the complete code, one line per row.
- **Line-by-line table** - every line of that listing, with what it does and, more usefully, why it is written that way.
- **Notes** - complexity, memory cost on the target MCU, and any defect found in that fragment.

A note on fidelity

The code in this document is the code from the source, transcribed faithfully. Where the source contains an error, the error is reproduced in the listing and then flagged in the line-by-line table and in Chapter 12, rather than silently corrected - you need to know what the original said. Corrected versions are given separately, and always marked as corrections.

NINE DEFECTS WERE FOUND DURING EXTRACTION

The source is a careful document, but consolidating it surfaces problems that are invisible when each chapter is read on its own: two sections give contradictory instructions about when to call `rigLStep()`; one memory figure is wrong by a factor of 250; the DROP and GROW steps as written do not preserve the exact-K conservation the algorithm depends on; and the gradient the GROW step needs is zeroed by another component before GROW can use it. Chapter 12 lists all nine with a proposed fix for each. Read it before you implement.

Build order

1. <code>findAbsKthSmallestActive()</code>	<code>MinMax.c</code>	no dependencies
2. <code>findAbsKthLargestInactive()</code>	<code>MinMax.c</code>	no dependencies
3. <code>weightMask</code> field	<code>Linear.h</code>	enables everything below
+--> 4. <code>mask-aware matmul</code>	<code>Matmul.c</code>	(forward speedup)
+--> 5. <code>mask-aware SGD</code>	<code>Sgd.c</code>	(keeps zeros at zero)
+--> 5b. <code>mask-aware AdamW</code>	<code>AdamW.c</code>	(optional optimiser)
6. <code>rigLStep()</code>	<code>RigL.c</code>	needs 1, 2, 3
7. <code>serializeSparsity()</code>	<code>Serialize.c</code>	persist the mask
8. <code>mask init + UserAPI</code>	<code>UserAPI.c</code>	create the mask
9. <code>training-loop integration</code>	<code>TrainingLoopApi.c</code>	

Figure 0.1 - Dependency order. Implement top to bottom; each step is testable on its own.

Contents

1. The RigL Algorithm	5
1.1 Why this works	5
1.2 The algorithm as stated in the source	5
1.3 The cosine schedule, and why it is shaped that way	6
1.4 Where it stands against dense training	6
2. What Exists, What Is Missing, and Where Each Piece Lives	7
2.1 Files this touches, and how much of each	7
3. Component 1 - findAbsKthSmallestActive()	9
3.1 Line by line	9
3.2 Worked example	10
3.3 Cost on the target	11
4. Component 2 - findAbsKthLargestInactive()	12
4.1 Line by line - only the differences from Component 1	12
4.2 Cost on the target	13
5. Component 3 - The weightMask Field	14
5.1 Line by line	14
5.2 Two struct definitions, one field	15
5.3 Memory cost	15
6. Component 4 - Mask-Aware Matmul	16
6.1 Line by line	16
6.2 Plumbing the mask into the kernel	16
6.3 Verifying it actually works	17
6.4 The gradient matmul must NOT be masked	17
7. Component 5 - Mask-Aware SGD	18
7.1 Line by line	18
8. Component 5b - Mask-Aware AdamW	20
8.1 Line by line - what differs from SGD	20
8.2 Memory cost of AdamW under sparsity	21
9. Component 6 - rigLStep(), the Algorithm Itself	22
9.1 Line by line	23
9.2 What one call actually does, in order	24
10. Component 7 - Persisting the Mask	25
10.1 Line by line	25
10.2 Wiring it into the tensor record	26
11. Components 8 to 10 - Initialization, Training Loop, Schedules	27
11.1 Component 8 - creating the mask (UserAPI)	27
11.2 Component 9 - the training loop	27
11.3 Component 10 - the two schedules	28

12. Verifying the Implementation	30
12.1 Unit tests, component by component	30
12.2 Integration checks during a real run	30
12.3 Results of a reference simulation	30
13. Ten Defects Found While Consolidating	32
13.1 Severity summary	32
13.2 D1 - The thresholds are off by one	32
13.3 D2 - Drop and grow are not disjoint	32
13.4 D3 - The gradients GROW needs may not exist	33
13.5 D4 - Contradictory ordering instructions	33
13.6 D5 - Stale momentum on regrowth	33
13.7 D6 - The AdamW memory figure is not achievable	33
13.8 D7 - Two struct definitions	33
13.9 D8 - malloc on the target	34
13.10 D9 - Two smaller correctness gaps	34
13.11 D10 - The complexity estimate uses the wrong n	34
14. Appendix - Consolidated RigL.h and RigL.c	36
14.1 RigL.h	36
14.2 RigL.c	36
15. Appendix - Where Every Fragment Came From	39

1. The RigL Algorithm

Source: ODT_Complete_Reference_v2.pdf, sections 2.1, 2.2, 2.3, 2.4 and 33.4

RigL - Rigging the Lottery - trains a sparse network from scratch in a single pass. It keeps a fixed number of active weights throughout training and periodically changes **which** weights are active: it drops the active weights that are smallest in magnitude, and grows the inactive weights whose gradients are largest.

1.1 Why this works

Signal	Used for	Reasoning
$ w[i] $ small, weight active	DROP it	A small weight contributes little to the output, so removing it costs little accuracy
$ g[i] $ large, weight inactive	GROW it	The gradient of an inactive weight says how much the loss WOULD fall if that connection were allowed to move. It is a direct estimate of the value of activating it
K dropped = K grown	Conservation	The active count never changes, so memory and compute stay fixed for the whole run - the property that makes RigL viable on an MCU

THE ONE IDEA

A pruned weight is not dead. Its gradient can still be computed, and that gradient tells you whether it deserves to come back. RigL is the algorithm that acts on this observation: magnitude decides what leaves, gradient decides what enters, and the two are balanced so the sparsity level never moves.

1.2 The algorithm as stated in the source

```

1  RIGL (per sparse layer, every T training steps):
2
3      alpha(t) = (0.3/2) * (1 + cos(pi * t / T_end))    // cosine decay
4      K        = floor( alpha(t) * (1 - s) * N )        // weights to swap
5
6      DROP K active weights with smallest |W|:
7          thresh = K-th smallest |W[i]| in the active set
8          for M[i]=1 and |W[i]| <= thresh: M[i]=0; W[i]=0; G[i]=0
9
10     GROW K inactive weights with largest |G|:
11         thresh = K-th largest |G[i]| in the inactive set
12         for M[i]=0 and |G[i]| >= thresh: M[i]=1; W[i]=0
13
14     Zero the gradients of all remaining inactive weights: G[i]=0

```

Listing 1.1 - The algorithm, transcribed from section 2.2 of the source.

Symbol	Meaning	Typical value for the HAR model
s	Target sparsity	0.9 - nine of every ten weights inactive
N	Total weights in the layer	73,728 for the 1152 x 64 linear layer
(1-s)*N	Active weights - constant for the whole run	7,373
alpha(t)	Fraction of active weights swapped at step t	0.30 falling to 0.00
K	Number swapped this step = alpha * active	2,212 at step 0, 0 after T_end
T	Steps between mask updates	100

Symbol	Meaning	Typical value for the HAR model
T_end	Step after which the mask is frozen	8,280 = 80% of 10,350

1.3 The cosine schedule, and why it is shaped that way

$$\alpha(t) = (\alpha_{\text{init}} / 2) * (1 + \cos(\pi * t / T_{\text{end}}))$$

t = 0 alpha = 0.30 K = 30% of active explore

t = T_end/2 alpha = 0.15 K = 15% of active consolidate

t = T_end alpha = 0.00 K = 0 mask frozen

t > T_end alpha = 0 only weight VALUES change, not positions

The mask must stop moving before training ends. If connectivity were still changing in the final epochs, the newly grown weights - which start at exactly zero - would never get enough updates to become useful, and every swap would cost accuracy instead of buying it. Freezing the mask at 80% of training leaves a fifth of the run to refine a fixed structure.

WHY IT PAIRS WITH A COSINE LEARNING RATE

Both schedules decay together, and they are saying the same thing at different levels. Early on: a high learning rate explores weight VALUES while a high alpha explores weight POSITIONS. Late on: a low learning rate refines values while alpha = 0 holds positions still. Running CosineAnnealingLR and the RigL alpha decay with the same T_max is not a coincidence of convenience - it keeps exploration and refinement aligned in both spaces.

1.4 Where it stands against dense training

Setting	Accuracy	Note
ResNet-50 / ImageNet, dense	76.8%	The reference
RigL, 80% sparse, 1x training	74.6%	2.2 points below dense at one fifth of the weights
RigL, 80% sparse, 5x training	76.4%	Matches dense - the accuracy is recoverable with a longer run
HAR, 90% sparse + FQT	~90-93% (estimate)	The target for this implementation; not yet measured

WHICH OF THESE NUMBERS ARE MEASUREMENTS

The ResNet-50 rows are published results quoted by the source. The HAR row is described in the source as an estimate, and it is repeated here as one. Nothing in this document reports a measurement taken on the STM32 target; when you run it, the numbers in Chapter 11 are the ones to fill in.

2. What Exists, What Is Missing, and Where Each Piece Lives

Source: ODT_Complete_Reference_v2.pdf, section 2.4 ('RigL: ODT Gap') and the 'Missing RigL Code' section of all 39 chapters

Before writing anything, it is worth knowing exactly how much of RigL the ODT library already provides. The answer is: the primitives exist, the algorithm does not.

Already in ODT	What it gives you
BOOL tensor type	A bit-packed tensor - the mask storage format, 1 bit per weight rather than 1 byte
tensorBoolGet / tensorBoolSet	Read and write one mask bit by flat index
bernoulliFillMask	Fill a BOOL tensor with independent Bernoulli draws - the random initial mask
calcNumberOfElementsByTensor	Element count, used by every loop in this document
XorShift32 RNG with seeding	Reproducible masks and reproducible stochastic rounding
Cosine LR scheduler	The same cosine math the alpha schedule needs

Missing - must be built	File	This document
findAbsKthSmallestActive()	MinMax.c	Chapter 3
findAbsKthLargestInactive()	MinMax.c	Chapter 4
weightMask field in linearConfig_t	Linear.h	Chapter 5
Mask-aware inner loop	Matmul.c	Chapter 6
Mask-aware parameter update	Sgd.c	Chapter 7
Mask-aware update with moment clearing	AdamW.c	Chapter 8
rigLStep()	RigL.h / RigL.c (new)	Chapter 9
serializeSparsity() / deserializeSparsity()	Serialize.c	Chapter 10
Mask creation for sparse layers	UserAPI.c	Chapter 11
rigLStep() call site and config fields	TrainingLoopApi.c	Chapter 11

SEVEN, EIGHT OR TEN COMPONENTS?

The source says seven. That count omits `deserializeSparsity()` - without which a saved mask cannot be loaded back, so checkpointing is only half implemented - and it omits the UserAPI and training-loop integration described in chapters 38 and 37, without which nothing ever calls `rigLStep()`. Counting honestly there are ten pieces of work, which is why this document has ten component chapters rather than seven.

2.1 Files this touches, and how much of each

File	Change	Size of change	Risk if wrong
MinMax.c / .h	Two new functions	~60 lines	Wrong threshold - drops or grows the wrong count
Linear.h	One struct field, one initialiser line	2 lines	None if NULL-defaulted; dense layers keep working
Matmul.c	One conditional in the hot loop	~4 lines	Silently wrong outputs if the flat index is miscomputed
Sgd.c	One conditional in the update loop	~5 lines	Inactive weights drift away from zero and sparsity is fake

File	Change	Size of change	Risk if wrong
AdamW.c	One conditional plus moment clearing	~8 lines	Stale moments cause a bad first step after a grow
RigL.c / .h	New file	~50 lines	The algorithm itself
Serialize.c	Two functions	~25 lines	Mask lost at checkpoint - the model reloads dense
UserAPI.c	Mask allocation for sparse specs	~8 lines	No mask means rigLStep() silently skips every layer
TrainingLoopApi.c	Config fields and one call	~10 lines	The mask never updates; you get static random sparsity

3. Component 1 - findAbsKthSmallestActive()

Source: ODT_Complete_Reference_v2.pdf, section 7.4 (primary) and section 39.2 (variant)

Purpose. Find the K-th smallest absolute weight value among the **active** weights of a layer. This single number is the DROP threshold: every active weight whose magnitude is at or below it will be deactivated.

Depends on: nothing. **Used by:** rigLStep() (Chapter 9). This is the first thing to implement and the easiest to unit-test.

```

1  float findAbsKthSmallestActive(tensor_t *weights,
2                                tensor_t *mask,
3                                size_t K) {
4      size_t n = calcNumberOfElementsByTensor(weights);
5      float *w = (float *)weights->data;
6
7      /* Pass 1: how many weights are currently active? */
8      size_t count = 0;
9      for (size_t i = 0; i < n; i++)
10         if (tensorBoolGet(mask, i)) count++;
11
12     if (K >= count) return 1e38f; /* keep all: threshold = +inf */
13
14     float *vals = malloc(count * sizeof(float));
15     if (!vals) { exit(1); }
16
17     /* Pass 2: gather |w| of the active weights only. */
18     size_t idx = 0;
19     for (size_t i = 0; i < n; i++)
20         if (tensorBoolGet(mask, i)) vals[idx++] = fabsf(w[i]);
21
22     /* Partial selection sort: sorts only the first K+1 positions. */
23     for (size_t i = 0; i <= K && i < count; i++) {
24         size_t minIdx = i;
25         for (size_t j = i + 1; j < count; j++)
26             if (vals[j] < vals[minIdx]) minIdx = j;
27         float tmp = vals[i];
28         vals[i] = vals[minIdx];
29         vals[minIdx] = tmp;
30     }
31
32     float thresh = vals[K < count ? K : count - 1];
33     free(vals);
34     return thresh;
35 }
```

Listing 3.1 - findAbsKthSmallestActive(), transcribed from section 7.4.

3.1 Line by line

Line	Code	What it does and why
1■3	float findAbsKth...(weights, mask, K)	Returns a float threshold. weights is the layer's weight tensor, mask the BOOL tensor of the same element count, K how many weights the caller wants to drop. The function does not modify anything - it only computes a number, which makes it trivially testable.
4	n = calcNumberOfElementsByTensor(weights)	Total elements in the layer, active and inactive. For the HAR 1152 x 64 layer this is 73,728. Note this is the count of ELEMENTS, not bytes - the mask is indexed by the same flat index.

Line	Code	What it does and why
5	<code>float *w = (float *)weights->data</code>	Direct float access, bypassing the tensor accessors. This is valid only because the weights are FLOAT32. For SYM_INT32 weights the stored values are int32 mantissas and you must reconstruct <code>mantissa * scale</code> before comparing magnitudes - see the note at the end of this chapter.
8■10	<code>for i: if (tensorBoolGet(mask,i)) count++</code>	First pass over the mask, counting active weights. This is needed because the buffer allocated on line 14 must be exactly the right size, and the count is not stored anywhere - the mask is the only record of it.
12	<code>if (K >= count) return 1e38f</code>	Guard for the degenerate case: the caller wants to drop at least as many weights as are active. Returning a huge threshold means every active weight satisfies <code> w <= thresh</code> , so all of them drop. It also protects line 31 from indexing past the end of the buffer. 1e38 is just under FLT_MAX (3.4e38), so it is a finite float that no real weight can exceed.
14	<code>vals = malloc(count * sizeof(float))</code>	A scratch buffer for the active magnitudes. The size is not known at compile time, which is why heap allocation is used - see the MCU warning below, because on a Cortex-M7 this is the most questionable line in the whole implementation.
15	<code>if (!vals) { exit(1); }</code>	Allocation failure handling. <code>exit(1)</code> is acceptable in a host-side prototype and is NOT acceptable on the target: there is no operating system to exit to. On the MCU this must return an error code and let the caller skip the RigL step for this layer.
18■20	<code>if (tensorBoolGet(mask,i)) vals[idx++] = fabsf(w[i])</code>	Second pass, gathering the absolute values of the active weights into the dense scratch buffer. <code>fabsf</code> not <code>fabs</code> - the float version, avoiding a double promotion that on an M7 costs both cycles and code size. After this loop <code>idx == count</code> .
22	<code>for (i = 0; i <= K && i < count; i++)</code>	The outer loop of a PARTIAL selection sort. A full sort would run to <code>count</code> ; stopping at K+1 is what makes this O(count*K) instead of O(count^2). The <code>i < count</code> term is belt and braces - line 12 already guarantees K < count.
23■25	<code>minIdx = i; for j > i: if smaller, minIdx = j</code>	Classic selection sort inner loop: scan the unsorted tail for the smallest remaining value. This is the hot loop of the function and the reason the cost is O(count*K).
26■28	<code>swap vals[i] and vals[minIdx]</code>	Move that smallest value into position i. After the outer loop finishes, positions 0..K hold the K+1 smallest magnitudes in ascending order; positions K+1.. are in arbitrary order, which is fine because they are never read.
31	<code>thresh = vals[K < count ? K : count - 1]</code>	The K-th smallest magnitude, zero-indexed - so this is actually the (K+1)-th smallest value. The ternary is redundant given line 12 but makes the function safe if that guard is ever changed. This line is where defect D1 in Chapter 12 lives: combined with the <code><=</code> comparison in <code>rigLStep()</code> , it drops K+1 weights rather than K.
32■33	<code>free(vals); return thresh;</code>	Release the scratch buffer and return. Note the buffer is freed on every path except the <code>exit(1)</code> on line 15, so there is no leak.

3.2 Worked example

The source gives this example in section 39.2. It is worth working through carefully, because it exposes a defect.

`active |w| = [0.01, 0.05, 0.03, 0.12, 0.08],    K = 2`

`after the partial sort: [0.01, 0.03, 0.05, ...]`

`thresh = vals[2] = 0.05`

`the source then states: 'weights with |w| <= 0.05 are dropped:`

`0.01 and 0.03. Exactly 2 weights dropped = K.'`

THE EXAMPLE UNDERCOUNTS BY ONE

The condition is $|w| \leq 0.05$ and the value 0.05 is itself in the active set - so THREE weights match: 0.01, 0.03 and 0.05. To drop exactly K you need either `thresh = vals[K-1]` with $\leq$, or `thresh = vals[K]` with a strict $<$. As written, DROP removes K+1 weights. The GROW step has the mirror-image error, so the two happen to cancel and total sparsity is preserved - but the swap size is not the K you asked for, and with duplicate magnitudes the cancellation stops being exact. Chapter 12, defect D1.

3.3 Cost on the target

Quantity	Value for the HAR 1152 x 64 layer at 90% sparsity
n (all weights)	73,728
count (active)	7,373
K at alpha = 0.3	2,212
Comparisons, $O(\text{count} \times K)$	$7,373 \times 2,212 = 16.3 \text{ M}$
Scratch memory	$7,373 \times 4 \text{ B} = 29 \text{ KB}$ on the heap
Frequency	Once per sparse layer, every 100 steps

THE SOURCE'S COMPLEXITY ESTIMATE IS OFF

Section 7.4 estimates the cost as $n \times K$ and gives $8192 \times 819 = 6.7 \text{ M}$ comparisons for a 128×64 layer. The sort runs over the ACTIVE values only, so the true cost is $\text{count} \times K$, which at 90% sparsity is an order of magnitude smaller than the n-based figure. The estimate is conservative in the safe direction, but if you are budgeting cycles, use count, not n. Chapter 12, defect D10.

MALLOC ON A CORTEX-M7

29 KB from the heap, allocated and freed every 100 steps, for every sparse layer, on a device with 320 KB of SRAM and no MMU. Repeated allocation of differently-sized blocks fragments the heap, and the failure mode is a NULL return at some unpredictable point hours into training. Two fixes: allocate one scratch buffer of the largest layer size ONCE at initialisation and reuse it, or replace the gather-and-sort with a two-pass histogram over magnitudes that needs only a fixed-size bin array. The first fix is a ten-line change and is what this document recommends.

4. Component 2 - findAbsKthLargestInactive()

Source: ODT_Complete_Reference_v2.pdf, section 7.4 (primary) and section 39.3 (variant)

Purpose. Find the K-th largest absolute **gradient** among the **inactive** weights. This is the GROW threshold: every inactive weight whose gradient magnitude is at or above it will be activated. It is the mirror image of Component 1 - inactive instead of active, gradients instead of weights, largest instead of smallest.

```

1  float findAbsKthLargestInactive(tensor_t *grads,
2                                tensor_t *mask,
3                                size_t K) {
4      size_t n = calcNumberOfElementsByTensor(grads);
5      float *g = (float *)grads->data;
6
7      size_t count = 0;
8      for (size_t i = 0; i < n; i++)
9          if (!tensorBoolGet(mask, i)) count++;
10
11     if (K >= count) return 0.0f;    /* grow all: threshold = 0 */
12
13     float *vals = malloc(count * sizeof(float));
14     if (!vals) { exit(1); }
15
16     size_t idx = 0;
17     for (size_t i = 0; i < n; i++)
18         if (!tensorBoolGet(mask, i)) vals[idx++] = fabsf(g[i]);
19
20     /* Partial selection sort, DESCENDING this time. */
21     for (size_t i = 0; i <= K && i < count; i++) {
22         size_t maxIdx = i;
23         for (size_t j = i + 1; j < count; j++)
24             if (vals[j] > vals[maxIdx]) maxIdx = j;
25         float tmp = vals[i];
26         vals[i] = vals[maxIdx];
27         vals[maxIdx] = tmp;
28     }
29
30     float thresh = vals[K < count ? K : count - 1];
31     free(vals);
32     return thresh;
33 }
```

Listing 4.1 - findAbsKthLargestInactive(), from section 7.4.

4.1 Line by line - only the differences from Component 1

Line	Code	What it does and why
1	tensor_t *grads	The first argument is the GRADIENT tensor, not the weights. This is the whole point of RigL: growth decisions are made on gradient magnitude, because the gradient of an inactive weight estimates how much the loss would improve if that connection were switched on.
5	float *g = (float *)grads->data	Direct float access to the gradients. In ODT, gradient tensors may be SYM_INT32 when FQT is enabled - in that case this cast is wrong and you must reconstruct the real value from the mantissa and scale before taking magnitudes. Chapter 12, defect D3b.
9	if (!tensorBoolGet(mask, i)) count++	Note the negation. Component 1 counted active weights; this counts INACTIVE ones. At 90% sparsity count here is about 66,355 - nine times larger than in Component 1, which makes this the more expensive of the two functions.

Line	Code	What it does and why
11	<code>if (K >= count) return 0.0f</code>	The degenerate guard, and note the return value is 0.0f, not 1e38f. Because the GROW comparison is <code> g >= thresh</code> , a threshold of zero matches every inactive weight, so all of them grow - the correct meaning of 'you asked for more than exist'. Returning +inf here, by symmetry with Component 1, would have grown nothing and silently densified the layer over time.
18	<code>vals[idx++] = fabsf(g[i])</code>	Gathers gradient for inactive positions. This is the line that depends on defect D3: if the gradient for inactive weights was never computed, or was zeroed by the previous <code>rigLStep()</code> , every value gathered here is 0.0 and the growth decision becomes arbitrary.
20■27	<code>if (vals[j] > vals[maxIdx]) maxIdx = j</code>	The only structural change from Component 1: <code>></code> instead of <code><</code> , and <code>maxIdx</code> instead of <code>minIdx</code> , giving a descending partial sort. Everything else about the sort is identical.
29	<code>thresh = vals[K < count ? K : count - 1]</code>	The K-th largest gradient magnitude, zero-indexed. As in Component 1, pairing this with a non-strict <code>>=</code> in <code>rigLStep()</code> activates K+1 weights rather than K.

WHY THE TWO GUARDS RETURN OPPOSITE EXTREMES

This asymmetry looks like an inconsistency and is in fact the correct design. The DROP guard returns +inf so that `|w| <= inf` matches everything; the GROW guard returns 0 so that `|g| >= 0` matches everything. Both mean 'process all candidates'. If you copy one function to make the other - which is the obvious way to write them - this is the line to check first.

4.2 Cost on the target

Quantity	Value for the HAR 1152 x 64 layer at 90% sparsity
count (inactive)	66,355 - nine times Component 1
K at alpha = 0.3	2,212
Comparisons, $O(\text{count} \times K)$	$66,355 \times 2,212 = 147 \text{ M}$
Scratch memory	$66,355 \times 4 \text{ B} = 259 \text{ KB}$ on the heap
Verdict	Infeasible as written on a 320 KB device

THIS IS THE COMPONENT THAT WILL NOT FIT

259 KB of scratch heap on a device with 320 KB of SRAM total, while the weights, gradients, activations and optimiser state are also resident, is not going to allocate. Component 2 is where the malloc approach stops being merely inelegant and becomes a blocker. The histogram alternative - two passes over the gradients accumulating counts into, say, 256 magnitude bins, then a partial scan to find the bin containing the K-th largest - needs about 1 KB and no allocation at all, at the cost of a threshold that is approximate to within one bin width. For a swap decision that is re-made every 100 steps, an approximate threshold is entirely acceptable. Chapter 12, defect D8.

5. Component 3 - The weightMask Field

Source: ODT_Complete_Reference_v2.pdf, section 11.4 (primary) and section 39.4 (variant)

Purpose. Give a linear layer somewhere to keep its mask. This is two lines of code and it is the keystone of the whole implementation: the matmul, the optimiser and rigLStep() all reach for this one field. **Depends on:** nothing. **Everything else depends on it.**

```

1  /* Linear.h - add one field to the config struct */
2  typedef struct linearConfig {
3      parameter_t *weights;
4      parameter_t *bias;
5      bool hasBias;
6      arithmetic_t arithType;
7      tensor_t *weightMask; /* NEW: NULL = dense, non-NULL = RigL */
8  } linearConfig_t;
9
10 /* Linear.c - in linearInitConfig() */
11 cfg->weightMask = NULL; /* dense by default */
12
13 /* To enable RigL on a layer, at model-build time: */
14 tensor_t *mask = createBoolTensor(outFeatures * inFeatures);
15 bernoulliFillMask(mask, 1.0f - targetSparsity); /* 0.1 => 90% sparse */
16 cfg->weightMask = mask;

```

Listing 5.1 - The weightMask field and its initialisation.

5.1 Line by line

Line	Code	What it does and why
7	tensor_t *weightMask;	A pointer to a BOOL tensor holding one bit per weight. It is a POINTER, not an embedded tensor, so that dense layers pay only 4 bytes for the null pointer rather than carrying an unused structure.
11	cfg->weightMask = NULL;	The single most important line for backward compatibility. Every consumer of this field tests <code>if (mask != NULL)</code> before using it, so a layer built by existing code behaves exactly as before and no existing test changes behaviour. Omitting this line leaves the pointer indeterminate and produces crashes that look random.
14	createBoolTensor(outFeatures * inFeatures)	One bit per weight, bit-packed. The element count must match the weight tensor exactly, because every consumer indexes both with the same flat index. For the 1152 x 64 layer: 73,728 bits = 9,216 bytes.
15	bernoulliFillMask(mask, 1.0f - targetSparsity)	Independent Bernoulli draws. Note the argument is the probability of being ACTIVE, so for 90% sparsity you pass 0.1, not 0.9 - an easy inversion to get wrong, and the symptom is a model that trains fine and is nine times slower than expected. Because the draws are independent, the realised active count varies slightly around 0.1 x N rather than being exactly 7,373.
16	cfg->weightMask = mask;	Attaching the mask is what switches the layer into sparse mode. Ownership is now ambiguous - see the note below.

TWO THINGS THE SOURCE LEAVES UNSTATED

OWNERSHIP: nothing says who frees this tensor. `linearConfig_t` already has an `ownsQuantizations` flag for exactly this kind of question, and the mask needs the same treatment or it leaks on every model teardown.

ENFORCEMENT AT INIT: `bernoulliFillMask` sets the mask bits but does not touch the weights, so immediately after initialisation 90% of the weights are inactive yet still hold their random initial values. Section 38.4 calls `zeroInactiveWeights()` to fix this and section 11.4 does not mention it. If you skip it, the first forward pass is dense-valued but mask-skipped, and your loss curve starts from the wrong place.

5.2 Two struct definitions, one field

Sections 11.4 and 39.4 both show `linearConfig_t`, and they disagree about the other fields:

Section 11.4 shows	Section 39.4 shows
weights, bias, hasBias, arithType	weights, bias, four separate arithmetic_t fields, two quantization_t pointers, two outputMode_t fields, ownsQuantizations

Only the second is plausible for a library that supports FQT - the four arithmetic fields correspond to the forward, weight-gradient, bias-gradient and loss-propagation paths. The lesson for you is practical: **do not retype the struct from either listing**. Open `Linear.h`, find `linearConfig_t`, and add the one field. Chapter 12, defect D7.

5.3 Memory cost

Item	Bytes	Note
Mask, 1152 x 64 layer	9,216	1 bit per weight, bit-packed
Mask, 64 x 6 output layer	48	Usually left dense - see below
Weight tensor, unchanged	294,912	Still stored densely, with zeros in the inactive positions
Overhead of masking	3.1%	9,216 / 294,912

SPARSE HERE MEANS SKIPPED, NOT ABSENT

The weight tensor keeps its full dense allocation; the mask only says which entries participate. So RigL as implemented here buys COMPUTE, not weight MEMORY - the 294 KB tensor is still 294 KB. Getting the memory back needs a compressed format such as CSR, which costs indices and destroys the $O(1)$ random access that the mask-aware matmul relies on. For a device where the bottleneck is cycles and energy rather than flash, dense-plus-mask is the right trade, but be clear about which resource you are actually saving before quoting a 10x figure to anyone.

DO NOT SPARSIFY THE LAST LAYER

The 64 x 6 classifier holds 384 weights - 0.5% of the model. Making it 90% sparse saves nothing measurable and removes connections to individual output classes, which is exactly where damage hurts most. The same argument applies to biases and normalisation parameters, and it is the standard practice from the pruning literature: leave the first and last layers dense.

6. Component 4 - Mask-Aware Matmul

Source: ODT_Complete_Reference_v2.pdf, section 8.4 (primary) and section 39.5 (variant)

Purpose. Skip the multiply-accumulate for inactive weights. This is the component that converts sparsity into speed: at 90% sparsity, nine of every ten inner-loop iterations exit early. **Depends on:** Component 3.

```

1  /* Matmul.c - inside matmulFloatCore(), innermost loop */
2  for (size_t i = 0; i < aColumns; i++) {
3
4      size_t flatIdx = rowIndex * aColumns + i;
5
6      if (weightMask != NULL &&
7          !tensorBoolGet(weightMask, flatIdx))
8          continue;          /* inactive: contributes nothing */
9
10     float aVal = readBytesAsFloat(&A->data[aByteIdx]);
11     float bVal = readBytesAsFloat(&B->data[bByteIdx]);
12     result += aVal * bVal;
13 }

```

Listing 6.1 - The masked inner loop, from section 8.4.

6.1 Line by line

Line	Code	What it does and why
2	for (i = 0; i < aColumns; i++)	The reduction loop of the matmul: for one output element, sum over the shared dimension. This is the hottest loop in the entire library - for the HAR layer it executes 73,728 times per sample - which is why it is the right place to spend a branch.
4	flatIdx = rowIndex * aColumns + i	Row-major flat index of the weight being touched. This line must match the mask's element ordering exactly. The source justifies it: weights are stored as [out_neurons, in_neurons] and transposed in O(1) before the matmul, so rowIndex is the output neuron and i the input neuron. If that transpose is ever changed to a real data movement, or the storage order flips, this index silently addresses the wrong mask bit and you get a model that trains to a plausible but wrong result.
6	if (weightMask != NULL && ...)	The null test comes FIRST, and C's short-circuit evaluation guarantees tensorBoolGet is never called on a null pointer. This single test is what lets one matmul serve both dense and sparse layers, at the cost of one predictable branch per iteration on dense layers.
7	!tensorBoolGet(weightMask, flatIdx)	Read one bit. Internally this is <code>(data[idx >> 3] >> (idx & 7)) & 1</code> - a load, a shift and a mask. On an M7 that is roughly 3-4 cycles against the 1-cycle fused multiply-add it is protecting, so the arithmetic only pays off when the skip rate is high. At 90% sparsity it pays handsomely; at 20% sparsity this code is SLOWER than dense.
8	continue;	Skip the multiply. Note what is not done here: no zero is added, no counter incremented. The inactive weight is exactly absent from the sum, which is arithmetically identical to multiplying by a stored zero but costs nothing.
10-11	readBytesAsFloat(&A->data[aByteIdx])	The library's alignment-safe float read, used instead of a direct pointer dereference because tensor data need not be 4-byte aligned. On the M7 an unaligned word load is legal but slower; on stricter cores it faults.
12	result += aVal * bVal;	The accumulation, now reached only by active weights.

6.2 Plumbing the mask into the kernel

The listing assumes `weightMask` is in scope inside `matmulFloatCore()`, and in the existing code it is not. The source offers two options and recommends one:

Option	How	Verdict
Thread the parameter	Add a <code>tensor_t *weightMask</code> argument to <code>matmulFloatCore()</code> and a new public <code>matmulFloat32TensorsWithMask()</code> that passes it; existing callers pass NULL	Recommended. Explicit, re-entrant, and the compiler finds every call site for you
Global context variable	Set a file-scope mask pointer before calling the existing function	Rejected. Not re-entrant, breaks under any future threading, and a forgotten reset silently applies one layer's mask to another

6.3 Verifying it actually works

THE INSTRUCTION COUNTER TEST

ODT exposes `getMatmulInstructionCounter()`. Run one forward pass dense and one masked at 90% sparsity on the same 64 x 1152 layer. Dense should report 73,728 multiply-accumulates; masked should report roughly 7,373. If masked still reports 73,728, the mask is not reaching the kernel - most likely you added the parameter but a wrapper is still passing NULL. If it reports something between the two, your flat index is misaligned with the mask and you are skipping the wrong weights, which is far worse than not skipping at all because the model will still appear to train.

6.4 The gradient matmul must NOT be masked

THE SINGLE MOST IMPORTANT WARNING IN THIS DOCUMENT

RigL grows connections using the gradients of INACTIVE weights. Those gradients come from the weight-gradient matmul, $dL/dW = \delta^T x$, which is a different call from the forward matmul. If you apply the mask to the weight-gradient computation as well - which looks consistent and saves more compute - then every inactive weight has a gradient of exactly zero, `findAbsKthLargestInactive()` returns 0, and the GROW step selects essentially arbitrary positions. The model will still train, the sparsity will still look correct, and the mask will be evolving at random: you will have implemented static sparse training with extra steps. MASK THE FORWARD MATMUL AND THE LOSS-PROPAGATION MATMUL; LEAVE THE WEIGHT-GRADIENT MATMUL DENSE. Chapter 12, defect D3.

This is also where RigL's real cost sits. The forward pass gets nine tenths cheaper, but the weight-gradient pass stays dense, so the end-to-end training speedup is well under the 10x that the sparsity figure suggests. The published algorithm accepts exactly this trade: dense gradients are the price of learning the connectivity.

7. Component 5 - Mask-Aware SGD

Source: ODT_Complete_Reference_v2.pdf, section 12.4 (primary) and section 39.6 (variant)

Purpose. Stop the optimiser from moving inactive weights away from zero. Without this, weight decay, momentum and any residual gradient will gradually give every 'pruned' weight a non-zero value, and the sparsity becomes a fiction that only the mask believes in. **Depends on:** Component 3.

```

1  static void sgdUpdateKernelMasked(tensor_t **op, size_t n,
2                                     tensor_t *rawOut, tensor_t *aux,
3                                     const void *ctxv) {
4      const sgdUpdateCtx_t *ctx = ctxv;
5      tensor_t *mask = ctx->weightMask;      /* NULL for dense layers */
6
7      float *param = (float *)op[0]->data;
8      float *grad  = (float *)op[1]->data;
9      float *out   = (float *)rawOut->data;
10     size_t nElem = calcNumberOfElementsByTensor(rawOut);
11
12     for (size_t i = 0; i < nElem; i++) {
13
14         if (mask != NULL && !tensorBoolGet(mask, i)) {
15             out[i] = 0.0f;
16             grad[i] = 0.0f;
17             continue;
18         }
19
20         float g = grad[i] + ctx->weightDecay * param[i];
21         out[i] = param[i] - ctx->lr * g;
22     }
23 }

```

Listing 7.1 - Mask-aware SGD update, merged from sections 12.4 and 39.6.

7.1 Line by line

Line	Code	What it does and why
1■3	static void sgdUpdateKernelMasked(...)	A kernel in the ODT sense: it is invoked through the executeOp funnel with an operand array rather than named tensors. static keeps it file-local. The source presents this as a NEW function alongside the existing sgdUpdateKernel ; adding the mask check to the existing one and letting NULL mean dense would avoid the duplication.
4■5	ctx->weightMask	The mask arrives through the optimiser context, not as an operand. That means sgdUpdateCtx_t needs a new field and whoever configures the optimiser must copy linearConfig_t.weightMask into it - a plumbing step the source does not spell out and which is easy to forget, with the symptom that everything compiles and no masking happens.
7■9	param, grad, out as float*	Three views of the same parameter: current value, its gradient, and where the new value is written. out and param may alias the same buffer for an in-place update; nothing here breaks if they do.
10	nElem = calcNumberOfElementsByTensor(rawOut)	The element count comes from the OUTPUT tensor, and the mask must have exactly this many bits. Mismatched counts read past the mask's storage - one of the few ways this code can corrupt memory.
14	if (mask != NULL && !tensorBoolGet(mask, i))	The same NULL-then-bit pattern as the matmul, so dense layers are unaffected.

Line	Code	What it does and why
15	<code>out[i] = 0.0f;</code>	Force the weight to exactly zero rather than merely leaving it. This is what makes the sparsity real: an inactive weight is not approximately zero from weight decay, it is bit-exactly 0.0f. That in turn means a serialized model compresses well and the mask-skipped matmul is provably equivalent to the dense one.
16	<code>grad[i] = 0.0f;</code>	Clear the gradient after use. In ODT gradients accumulate across micro-batches, so without this the inactive positions would carry stale sums forward. But note the tension with RigL: this line runs on every optimiser step, while GROW needs the inactive gradients at the <code>rigLStep()</code> boundary. The two only coexist because <code>rigLStep()</code> runs BEFORE the optimiser in the same iteration - see Chapter 11 and defect D4.
17	<code>continue;</code>	Skip the arithmetic entirely. Unlike the matmul, the saving here is negligible - the optimiser runs once per step, not once per MAC. This branch is about CORRECTNESS, not speed.
20	<code>g = grad[i] + ctx->weightDecay * param[i]</code>	Coupled L2 weight decay, folded into the gradient. Note this is the coupled form, not AdamW's decoupled one; for plain SGD the two are equivalent up to a factor of the learning rate.
21	<code>out[i] = param[i] - ctx->lr * g</code>	The SGD step. Momentum, if configured, is applied elsewhere in the ODT pipeline - which raises a question the source does not answer: the momentum buffer for an inactive weight is never cleared here. Chapter 12, defect D5.

WHY FORCING THE WEIGHT TO ZERO MATTERS MORE THAN IT LOOKS

Suppose you skip line 15 and simply `continue` without writing. The inactive weight keeps whatever value it had. The masked matmul still ignores it, so the forward pass is unchanged and every test passes. Then you checkpoint the model, and on reload the mask is gone or misaligned - and suddenly those weights are live again with values that were never trained. Writing the zero makes the invariant 'inactive implies zero' hold in the DATA, not just in the mask, and the data is what survives serialization.

8. Component 5b - Mask-Aware AdamW

Source: ODT_Complete_Reference_v2.pdf, section 13.4

Purpose. The same protection as Component 5, plus one extra step that SGD does not need: clearing the first and second moment buffers for inactive weights. **Depends on:** Component 3. **Optional** - needed only if you train with AdamW rather than SGD.

```

1  for (size_t i = 0; i < n; i++) {
2
3      if (mask != NULL && !tensorBoolGet(mask, i)) {
4          param[i] = 0.0f;
5          grad[i]  = 0.0f;
6          m[i]     = 0.0f;      /* clear first-moment history */
7          v[i]     = 0.0f;      /* clear second-moment history */
8          continue;
9      }
10
11     float g = grad[i];
12     m[i] = beta1 * m[i] + (1.0f - beta1) * g;
13     v[i] = beta2 * v[i] + (1.0f - beta2) * g * g;
14     param[i] -= lrCorr * m[i] / (sqrtf(v[i]) + eps);
15     param[i] -= lr * weightDecay * param[i];
16     grad[i] = 0.0f;
17 }

```

Listing 8.1 - Mask-aware AdamW, from section 13.4.

8.1 Line by line - what differs from SGD

Line	Code	What it does and why
6■7	<code>m[i] = 0.0f; v[i] = 0.0f;</code>	The reason this component exists. Consider a weight that RigL drops at step 1000 and regrows at step 1500. Without clearing, its <code>m</code> and <code>v</code> still hold the moving averages from before it was dropped. On the first step after regrowth, Adam divides a stale <code>m</code> by the square root of a stale <code>v</code> and applies a full-sized update to a weight that was just reset to zero - a large, arbitrary jump in a direction computed from history that no longer applies.
6	<code>m[i] = 0.0f</code>	Clearing the first moment means the regrown weight's first update is driven purely by its current gradient, exactly as if the parameter were newly initialised. Which it is.
7	<code>v[i] = 0.0f</code>	Clearing the second moment has a subtler effect: because Adam divides by $\sqrt{v}$ with bias correction, a <code>v</code> of zero makes the first step after regrowth approximately $\pm$ the learning rate, the same size any fresh parameter would take. A stale large <code>v</code> would instead make the weight nearly frozen.
12■13	<code>m and v updates</code>	The standard exponential moving averages of the gradient and its square. Reached only by active weights.
14	<code>param[i] -= lrCorr * m[i] / (sqrtf(v[i]) + eps)</code>	The Adam step. <code>lrCorr</code> folds in the bias corrections for both moments so the division happens once rather than twice per element - worth doing on an M7 where a float divide is 14 cycles.
15	<code>param[i] -= lr * weightDecay * param[i]</code>	DECOUPLED weight decay applied to the parameter directly, not to the gradient - this is the <code>W</code> in AdamW, and it is a separate line rather than folded into <code>g</code> precisely so that the adaptive scaling does not divide it away.
16	<code>grad[i] = 0.0f</code>	Reset the accumulator, as in SGD, with the same interaction with the GROW step noted in Chapter 7.

ALSO CLEAR THE SGD MOMENTUM BUFFER

Section 13.4 correctly identifies stale moments as a hazard for AdamW and says nothing about SGD with momentum, which has exactly the same problem in a milder form: a regrown weight inherits the velocity it had before it was dropped. If your SGD carries a momentum buffer, clear that entry too, in the branch on line 14-18 of Listing 7.1. Chapter 12, defect D5.

8.2 Memory cost of AdamW under sparsity

Tensor	Elements	Bytes	Note
Weights	73,728	294,912	Dense storage regardless of mask
Gradients	73,728	294,912	Must stay dense - GROW needs them
m (first moment)	73,728	294,912	Allocated densely
v (second moment)	73,728	294,912	Allocated densely
Mask	73,728 bits	9,216	3% overhead
Total for one layer	-	1,188,864	1.13 MB - exceeds the 320 KB SRAM on its own

ADAMW PLUS THIS LAYER DOES NOT FIT ON THE TARGET

Section 13.4 states the AdamW cost as 88 KB by counting only the 7,373 ACTIVE weights. That figure is right only if the moment tensors are stored sparsely - and nothing in the implementation does that; **m** and **v** are allocated with the same dense shape as the parameters. The honest number for this layer is 1.13 MB, which does not fit. Either use SGD (which needs no moment buffers and brings the layer to 599 KB - still too large), or shrink the layer, which is what the source itself recommends in section 39.10 when it suggests a Conv1d feature extractor ahead of a much smaller linear head. Chapter 12, defect D6.

9. Component 6 - rigLStep(), the Algorithm Itself

Source: ODT_Complete_Reference_v2.pdf, section 39.7, with the DROP/GROW logic cross-checked against sections 2.2 and 33.4

Purpose. Everything so far has been scaffolding. This function is RigL: it computes the swap size from the cosine schedule, drops the weakest active weights, grows the most promising inactive ones, and leaves the layer with the same number of active weights it started with. **Depends on:** Components 1, 2 and 3.

```

1  void rigLStep(layer_t **model, size_t numLayers,
2              float sparsityTarget, size_t step, size_t totalSteps) {
3
4      /* Cosine decay of the swap fraction. */
5      float prog = (float)step / (float)(totalSteps > 0 ? totalSteps : 1);
6      float alpha = 0.3f * 0.5f * (1.0f + cosf(3.14159265f * prog));
7
8      for (size_t l = 0; l < numLayers; l++) {
9
10         if (model[l]->type != LINEAR) continue;
11
12         linearConfig_t *cfg = model[l]->config->linear;
13         tensor_t *mask = cfg->weightMask;
14         if (mask == NULL) continue;          /* dense layer */
15
16         tensor_t *weights = cfg->weights->param;
17         tensor_t *grads = cfg->weights->grad;
18         size_t n = calcNumberOfElementsByTensor(weights);
19         float *w = (float *)weights->data;
20         float *g = (float *)grads->data;
21
22         size_t numActive = 0;
23         for (size_t i = 0; i < n; i++)
24             if (tensorBoolGet(mask, i)) numActive++;
25
26         size_t K = (size_t)(alpha * (float)numActive);
27         if (K == 0) continue;                /* nothing to swap */
28
29         /* ---- DROP: deactivate the K smallest |w| ---- */
30         float dropThresh = findAbsKthSmallestActive(weights, mask, K);
31         for (size_t i = 0; i < n; i++) {
32             if (tensorBoolGet(mask, i) && fabsf(w[i]) <= dropThresh) {
33                 tensorBoolSet(mask, i, false);
34                 w[i] = 0.0f;
35             }
36         }
37
38         /* ---- GROW: activate the K largest |g| among inactive ---- */
39         float growThresh = findAbsKthLargestInactive(grads, mask, K);
40         for (size_t i = 0; i < n; i++) {
41             if (!tensorBoolGet(mask, i) && fabsf(g[i]) >= growThresh) {
42                 tensorBoolSet(mask, i, true);
43                 w[i] = 0.0f;                /* grow at zero */
44             }
45         }
46
47         /* ---- Zero the gradients of still-inactive weights ---- */
48         for (size_t i = 0; i < n; i++)
49             if (!tensorBoolGet(mask, i)) g[i] = 0.0f;
50     }
51 }

```

Listing 9.1 - rigLStep(), transcribed from section 39.7.

9.1 Line by line

Line	Code	What it does and why
1■2	<code>rigLStep(model, numLayers, sparsityTarget, step, totalSteps)</code>	Operates on the whole model, not one layer, so a single call updates every sparse layer. Note <code>sparsityTarget</code> is accepted and NEVER USED in the body - the actual sparsity is whatever the mask already encodes, and the function only preserves it. That is not a bug, but it is a misleading signature: a reader assumes passing 0.9 sets the sparsity, when in fact it is set once at initialisation by <code>bernoulliFillMask()</code> .
5	<code>prog = step / (totalSteps > 0 ? totalSteps : 1)</code>	Training progress in [0, 1]. The ternary guards against a divide-by-zero when the caller does not know the total step count. Both operands are cast to float first; integer division here would make prog 0 for the entire run.
6	<code>alpha = 0.3f * 0.5f * (1 + cosf(pi * prog))</code>	The cosine decay. At prog = 0, cos(0) = 1 so alpha = 0.3; at prog = 1, cos(pi) = -1 so alpha = 0. Two things are hard-coded that should not be: the initial alpha of 0.3, and the assumption that the mask freezes at the END of training. Section 2.3 specifies T_end = 80% of total steps, so the caller must pass <code>totalSteps * 0.8</code> here rather than the true total - an easy mistake that leaves the mask still swapping during the final epochs. Chapter 12, defect D9.
10	<code>if (model[l]->type != LINEAR) continue;</code>	Only linear layers are considered. Conv1d layers are skipped entirely even if section 20.4 describes a <code>kernelMask</code> for them - so on a conv-plus-linear model, RigL touches only the classifier head.
13■14	<code>mask = cfg->weightMask; if (mask == NULL) continue;</code>	The dense-layer escape. Together with line 10 this means <code>rigLStep()</code> is safe to call on any model, including a fully dense one, where it does nothing at all. That is a good property: the training loop needs no conditional.
16■17	<code>weights = cfg->weights->param; grads = cfg->weights->grad;</code>	The parameter and its gradient live in the same <code>parameter_t</code> . RigL needs both: weights for the DROP decision, gradients for GROW.
19■20	<code>float *w, *g</code>	Direct float access again, with the same FLOAT32-only limitation noted in Chapter 3. Under FQT these tensors may be SYM_INT32.
22■24	<code>count numActive</code>	A third full pass over the mask, after the two that find <code>AbsKthSmallestActive()</code> will do internally. Caching this count in the layer config would remove three O(n) passes per RigL step, though at 100-step intervals it hardly matters.
26	<code>K = (size_t)(alpha * (float)numActive)</code>	The swap size: a fraction of the ACTIVE weights, matching section 2.2's <code>alpha * (1-s) * N</code> since numActive is (1-s)*N. The cast truncates toward zero, which is the intended floor().
27	<code>if (K == 0) continue;</code>	Once alpha decays far enough that K rounds to zero, the layer is frozen. This is how the mask stops evolving without any explicit end-of-schedule test.
30	<code>dropThresh = findAbsKthSmallestActive(weights, mask, K)</code>	Component 1. Computed BEFORE the loop that uses it, so the threshold reflects the mask as it was at the start of this step.
32	<code>if (tensorBoolGet(mask,i) && fabsf(w[i]) <= dropThresh)</code>	The DROP test. Non-strict <code><=</code> combined with a zero-indexed K-th value drops K+1 weights, and drops MORE than that when magnitudes tie - which is common, because every weight that was grown but never updated is still exactly 0.0. Chapter 12, defect D1.
33■34	<code>tensorBoolSet(mask,i,false); w[i] = 0.0f;</code>	Deactivate and zero. Writing the zero here rather than waiting for the optimiser means the invariant holds immediately, which matters because the GROW loop that follows reads these same weights.
39	<code>growThresh = findAbsKthLargestInactive(grads, mask, K)</code>	Component 2 - and note WHEN it is called: after the DROP loop has already modified the mask. The weights just dropped are now inactive, so they are candidates for immediate regrowth. If one of them has a large gradient - which is entirely possible, since a weight can be small in magnitude yet have a steep gradient - it is dropped and regrown in the same step, wasting part of the swap budget and resetting a trained weight to zero for no benefit. The published algorithm excludes the just-dropped set from growth candidates. Chapter 12, defect D2.

Line	Code	What it does and why
41	<code>if (!tensorBoolGet(mask,i) && fabsf(g[i]) >= growThresh)</code>	The GROW test, with the mirror of defect D1: non-strict $\geq$ grows $K+1$. Because DROP also removes $K+1$, the total active count is preserved - the two errors cancel. That is luck, not design, and the cancellation fails as soon as ties are unevenly distributed.
42■43	<code>tensorBoolSet(mask,i,true); w[i] = 0.0f;</code>	Activate at zero, exactly as the RigL paper specifies. A grown connection starts with no contribution and must earn its value from gradients - which is why the mask has to stop changing well before the end of training, or late arrivals never get the chance.
47■48	<code>if (!tensorBoolGet(mask,i)) g[i] = 0.0f;</code>	Clear the gradients of everything still inactive. This prevents stale gradient values from influencing the NEXT <code>rigLStep()</code> - without it, a weight that had a large gradient a thousand steps ago would keep looking attractive forever.

9.2 What one call actually does, in order

step 900, alpha = 0.28, layer has 7,373 active of 73,728

1. $K = \text{floor}(0.28 * 7373) = 2,064$
2. $\text{dropThresh} = 2064\text{-th smallest } |w| \text{ among the } 7,373 \text{ active}$
3. DROP : ~2,065 weights mask 1->0, $w := 0$
active: 7,373 -> 5,308
4. $\text{growThresh} = 2064\text{-th largest } |g| \text{ among the now-}68,420 \text{ inactive}$
5. GROW : ~2,065 weights mask 0->1, $w := 0$
active: 5,308 -> 7,373 <- conserved
6. zero $g[i]$ for all 66,355 still-inactive weights

net effect: 2,065 connections have moved to better positions,
the active count is unchanged, and 2,065 weights
that were carrying trained values are now zero.

Figure 9.1 - One `rigLStep()` on the HAR layer at step 900.

THE COST NOBODY MENTIONS

Every swap throws away a trained weight and installs a zero. At $\alpha = 0.3$ that is 28% of the layer's learned values discarded in a single step. This is why the cosine decay is not optional and why the interval T matters: swap too often, or keep α high for too long, and the network spends the whole run relearning connections instead of converging. If your loss spikes at every multiple of 100 steps and does not recover before the next one, α is too high or T is too small.

10. Component 7 - Persisting the Mask

Source: ODT_Complete_Reference_v2.pdf, section 17.5 (both functions) and section 39.8 (write only)

Purpose. Save the mask with the checkpoint and load it back. After 50 epochs the mask IS the learned architecture - which 10% of connections matter - and a checkpoint without it restores a model that is dense, wrong, and 10x slower. **Depends on:** Component 3.

```

1 void serializeSparsity(tensor_t *mask, FILE *fp) {
2
3     if (mask == NULL || mask->quantization->type != BOOL) {
4         uint8_t noMask = 0;
5         fwrite(&noMask, 1, 1, fp);
6         return;
7     }
8
9     uint8_t hasMask = 1;
10    fwrite(&hasMask, 1, 1, fp);
11
12    size_t n = calcNumberOfElementsByTensor(mask);
13    size_t bytes = (n + 7) / 8;
14    fwrite(mask->data, 1, bytes, fp);
15 }
16
17 tensor_t *deserializeSparsity(size_t n, FILE *fp) {
18
19     uint8_t hasMask = 0;
20     fread(&hasMask, 1, 1, fp);
21     if (!hasMask) return NULL; /* dense layer */
22
23     tensor_t *mask = allocBoolTensor(n);
24     size_t bytes = (n + 7) / 8;
25     fread(mask->data, 1, bytes, fp);
26     return mask;
27 }

```

Listing 10.1 - Mask serialization, from section 17.5.

10.1 Line by line

Line	Code	What it does and why
3	if (mask == NULL quantization->type != BOOL)	Two guards in one: no mask at all, or a tensor that is not a bit mask. The second catches the case where some other tensor type is attached by mistake - it degrades to 'no mask' rather than writing garbage of the wrong length into the checkpoint.
4■6	write a zero byte and return	The presence flag. Writing one byte even in the dense case is what keeps the file format self-describing: the reader always consumes exactly one byte here and knows from its value whether more follows.
9■10	write a one byte	Mask present. Note the flag is written BEFORE the data, so a reader streaming the file never has to seek.
12	n = calcNumberOfElementsByTensor(mask)	The number of BITS, since a BOOL tensor's element count is its bit count.
13	bytes = (n + 7) / 8	Ceiling division: 73,728 bits gives exactly 9,216 bytes; 73,730 bits would give 9,217 with the last byte partly unused. The idiom $(n+7)/8$ avoids floating point and is exact for all n.

Line	Code	What it does and why
14	<code>fwrite(mask->data, 1, bytes, fp)</code>	Write the packed bits verbatim. This makes the checkpoint endian- and layout-dependent: it stores the in-memory bit order directly, so a file written by the host trainer is only portable to the MCU if both pack bits identically. Since ODT defines <code>tensorBoolGet</code> as <code>(data[i>>3] >> (i&7)) & 1</code> on both, they do - but the assumption is undocumented and worth a comment in the code.
18	<code>tensor_t *deserializeSparsity(size_t n, FILE *fp)</code>	The reader takes <code>n</code> as an argument because the bit count is not stored in this record - it is implied by the weight tensor that precedes it. That coupling is fragile but compact; the alternative is four more bytes per layer.
21■22	<code>read the flag; return NULL if zero</code>	NULL is the correct 'dense' value, matching the default set in Component 3 - so a dense checkpoint loads into a dense layer with no special case.
24	<code>mask = allocBoolTensor(n)</code>	Allocate before reading. The caller becomes the owner and must attach it to <code>cfg->weightMask</code> and eventually free it - the same ownership question raised in Chapter 5, now with a second place to leak.
26	<code>fread(mask->data, 1, bytes, fp)</code>	Read the packed bits back. Neither this call nor the <code>fwrite</code> above checks its return value; a truncated file yields a partly-initialised mask and no error. On an embedded target reading from an SD card, that check is not optional.

10.2 Wiring it into the tensor record

```

1 void serializeTensor(tensor_t *t, tensor_t *mask, FILE *f) {
2     serializeShape(t->shape, f);
3     serializeQuantization(t->quantization, f);
4     serialWriteBytes(t->data, calcBytesPerTensor(t), f);
5     serializeSparsity(mask, f);          /* now saves the mask */
6 }
7
8 /* called from serializeLinear() as: */
9 serializeTensor(weights, cfg->weightMask, f);

```

Listing 10.2 - Threading the mask through `serializeTensor()`, from section 17.5.

THIS CHANGES A FORMAT THE SOURCE CALLS 'LOCKED'

Chapter 17 of the source describes the checkpoint format as 'Locked Binary Format v2', and adding a presence byte to every tensor record changes it for DENSE models too - old files no longer parse, and old readers cannot read new files. If the format really is locked, this belongs behind a version bump to v3, with the reader accepting both. The change is one byte per tensor, so the cost of doing it properly is trivial; the cost of not doing it is a checkpoint corpus that silently splits in two. Chapter 12, defect D9.

What is stored	Bytes for the HAR 1152 x 64 layer
Weight data	294,912
Presence flag	1
Packed mask	9,216
Mask overhead	3.1% of the record

11. Components 8 to 10 - Initialization, Training Loop, Schedules

Source: ODT_Complete_Reference_v2.pdf, sections 9.4, 14.4, 37.4, 38.4 and 39.9

11.1 Component 8 - creating the mask (UserAPI)

Nothing built so far ever creates a mask. `buildSequentialModel()` must do it when a layer spec asks for sparsity.

```

1  /* UserAPI.c - inside buildSequentialModel(), per LINEAR layer spec */
2  if (spec->sparse) {
3      size_t maskBits = inFeat * outFeat;
4      tensor_t *mask = createBoolTensor(maskBits);
5      bernoulliFillMask(mask, 1.0f - spec->sparsity);
6      zeroInactiveWeights(linearCfg, mask);
7      linearCfg->weightMask = mask;
8  }
```

Listing 11.1 - Mask creation, from section 38.4.

Line	Code	What it does and why
2	<code>if (spec->sparse)</code>	A new field on <code>layerSpec_t</code> , alongside a <code>float sparsity</code> . Layers that do not ask for sparsity are untouched and keep <code>weightMask = NULL</code> .
3	<code>maskBits = inFeat * outFeat</code>	One bit per weight. This must equal <code>calcNumberOfElementsByTensor(weights)</code> exactly - if the layer has a bias, it is NOT masked, and the bias is a separate tensor, so the product of the two feature counts is right.
4	<code>createBoolTensor(maskBits)</code>	Allocates <code>ceil(bits/8)</code> bytes of bit-packed storage plus the tensor header.
5	<code>bernoulliFillMask(mask, 1.0f - spec->sparsity)</code>	The argument is P(active). For <code>spec->sparsity = 0.9</code> this passes 0.1. Because the draws are independent, the realised active count is binomial around 7,373 with a standard deviation of about 81 - so your first log line will read 7,412 active, not 7,373, and that is correct behaviour, not a bug.
6	<code>zeroInactiveWeights(linearCfg, mask)</code>	Enforces the invariant at initialisation: every inactive weight is set to exactly zero. Section 11.4 omits this call and section 38.4 includes it; without it, inactive positions keep their random init values, which then appear in the checkpoint and come back to life if the mask is ever lost.
7	<code>linearCfg->weightMask = mask;</code>	Attach. From this point the matmul, the optimiser and <code>rigLStep()</code> all see the layer as sparse.

SEED BEFORE YOU FILL

Section 9.4 makes a point worth repeating: call `rngSetSeed()` before `bernoulliFillMask()` AND before the training loop. The mask draw consumes RNG state that stochastic rounding would otherwise have used, so the two are coupled - the same seed gives you the same mask and the same rounding sequence only if the ORDER of operations is also the same. This is the difference between an experiment you can reproduce and one you cannot.

11.2 Component 9 - the training loop

```

1  #define RIGL_INTERVAL 100          /* update the mask every 100 steps */
2
3  size_t step = 0;
4  for (size_t epoch = 0; epoch < epochs; epoch++) {
5
6      for (size_t i = 0; i < trainSize; i++) {
7
```

```

8      batch_t *b = loader.getBatch(&loader, i);
9
10     forward(model, b->input);
11     backward(model, b->label);
12
13     if (step % RIGL_INTERVAL == 0)
14         rigLStep(model, nLayers, 0.9f, step, totalSteps);
15
16     optimizerFunctions[SGD_M].step(&optim);
17     step++;
18 }
19
20 lrSchedulerStep(&sched);
21 }

```

Listing 11.2 - The training loop, from section 39.9.

Line	Code	What it does and why
1	#define RIGL_INTERVAL 100	T from the algorithm. Over a 10,350-step run this gives 103 mask updates. Too small and the network never settles between swaps; too large and the connectivity barely explores.
10■11	forward(); backward();	The gradients that rigLStep() will read are produced here. The backward pass must produce gradients for INACTIVE weights too - see Chapter 6 - or the GROW step has nothing to rank.
13■14	if (step % RIGL_INTERVAL == 0) rigLStep(...)	The mask update, placed AFTER backward and BEFORE the optimiser step.
16	optimizerFunctions[SGD_M].step(&optim)	The mask-aware update from Component 5. Because it runs after rigLStep(), the weights just grown get their first update in this very iteration rather than waiting a full step.
21	lrSchedulerStep(&sched)	Learning-rate decay, once per epoch, independent of the alpha schedule inside rigLStep().

THE SOURCE CONTRADICTS ITSELF ABOUT THIS ORDERING

Section 37.4 states: 'rigLStep() must be called AFTER the optimizer step but BEFORE the next forward pass. The order matters.' Section 39.9 states the opposite: 'Run rigLStep() BEFORE the optimizer step. If rigLStep() runs AFTER the optimizer, the new weights miss their first update.' The code in 39.9 implements the second. Both cannot be right, and the difference is observable: it decides whether a grown weight waits 0 or 100 steps for its first gradient, and whether the gradients rigLStep() reads have already been zeroed by the optimiser (Component 5 line 16 clears them). BEFORE the optimiser is the correct choice, for exactly the reason 39.9 gives, and because the optimiser would otherwise have wiped the inactive gradients that GROW depends on. Chapter 12, defect D4.

11.3 Component 10 - the two schedules

The learning-rate schedule and the alpha schedule are separate cosines that should share a horizon.

```

1  cosineAnnealingLrInit(&sched, &optimizer,
2                          0.01f,          /* eta_max */
3                          totalEpochs,
4                          1e-5f);         /* eta_min */
5
6  /* alpha(step) = (0.3/2) * (1 + cos(pi * step / T_end)) */
7  /* with T_end = 0.8 * totalSteps */

```

Listing 11.3 - Learning rate and alpha, from section 14.4.

Step	Epoch	alpha	K (of 7,373 active)	Learning rate	Phase
0	0	0.300	2,212	0.010	Explore both values and connectivity
4,140	20	0.150	1,106	~0.006	Consolidate

Step	Epoch	alpha	K (of 7,373 active)	Learning rate	Phase
8,280	40	0.000	0	~0.001	Mask frozen
10,350	50	0.000	0	1e-5	Refine values only

PASS T_END, NOT TOTALSTEPS

rigLStep() computes $\text{prog} = \text{step} / \text{totalSteps}$ and reaches $\alpha = 0$ only when $\text{prog} = 1$, i.e. at the very last step. The schedule the source specifies wants $\alpha = 0$ at 80% of training. So the caller must pass $0.8f * \text{totalSteps}$ as the `totalSteps` argument - the parameter is misnamed, and passing the true total leaves the mask swapping into the final epoch, which is precisely what the schedule exists to prevent. Chapter 12, defect D9.

12. Verifying the Implementation

Each component has a test that isolates it. Run them in build order; a failure at step k with steps $1..k-1$ passing localises the bug to one function.

12.1 Unit tests, component by component

Component	Test	Expected
1 - kth smallest	Mask with 5 known active weights [0.01,0.05,0.03,0.12,0.08], $K=2$	Returns 0.05. Then count how many satisfy ≤ 0.05 : you should get 3, which demonstrates defect D1
1 - guard	$K = \text{count}$	Returns 1e38, and every active weight drops
2 - kth largest	Inactive gradients [0.5,0.1,0.9,0.3], $K=1$	Returns 0.5
2 - guard	$K = \text{count}$	Returns 0.0, and every inactive weight grows
3 - default	Build a model with no sparse specs	Every weightMask is NULL; all existing tests still pass unchanged
4 - matmul	Dense vs 90%-masked forward on identical inputs	Instruction counter falls from 73,728 to about 7,373; outputs differ only in the masked contributions
4 - equivalence	Zero the inactive weights, then run the DENSE matmul	Bit-identical to the masked result - this is the strongest single check that your flat index is right
5 - SGD	One step with 90% of the mask clear	Every inactive weight is exactly 0.0f afterwards, and its gradient is 0.0f
6 - rigLStep	Count active before and after one call	Equal, or differing only by the tie effects of D1
6 - schedule	Call at step 0, $T_{\text{end}}/2$ and T_{end}	$\alpha = 0.30, 0.15, 0.00$ and K scales with it
7 - round trip	serializeSparsity then deserializeSparsity	Every one of the 73,728 bits matches

12.2 Integration checks during a real run

Log these every epoch and watch their shape

- ☐ Active weight count per sparse layer. It must be FLAT. A downward drift means GROW is finding fewer candidates than DROP removes - usually because inactive gradients are zero (defect D3).
- ☐ Number of mask bits that changed since the last RigL step. It should start near K and fall to zero as α decays. If it is zero from the beginning, K is rounding to zero or the mask is NULL.
- ☐ Fraction of swaps that are re-grows of weights dropped in the same step. Should be small; a large fraction is defect D2.
- ☐ Training loss around each RigL step. A spike is expected; failure to recover before the next step means α is too high or T too small.
- ☐ Matmul instruction count. Should stay near $(1 - \text{sparsity}) \times \text{dense}$ for the whole run.
- ☐ Peak heap usage, if you kept the malloc approach. This is where the run dies on the target.

12.3 Results of a reference simulation

The consolidated code in Appendix A was compiled against stub headers and run for 8,200 steps on a synthetic 8,192-weight layer initialised at 10% density, with fresh random dense gradients supplied at each RigL step. This exercises the mask dynamics only - there is no forward pass, no loss and no optimiser - so it says nothing about accuracy. What it does test is every invariant this document claims.

Check	Result
Active count at initialisation	828 of 8,192 (10.1%) - binomial scatter around the requested 819, as expected
Active count after every one of 83 mask updates	828. Conservation held exactly at every step
Inactive weights holding a non-zero value at the end	0
alpha at steps 0 / 4,000 / 7,600	0.3000 / 0.1500 / 0.0018 - the cosine schedule
Behaviour after tEnd = 8,000	K = 0 and the mask stops changing

ONE FINDING THE SIMULATION SURFACED

The DROP step consistently removed 249 weights while K fell from 248 to 1. The cause is instructive: grown weights are set to exactly 0.0, and in this simulation nothing ever moves them, so after the first update hundreds of active weights share the identical magnitude 0.0. No threshold can separate a tied group, so any threshold at or above zero takes all of them. In a real run the optimiser moves grown weights off zero within one step and the ties disappear - but this is exactly the tie pathology behind defect D1, made visible. It is also why the Appendix A code grows **dropped** weights rather than **K**: whatever DROP actually removed, GROW restores the same number, so conservation survives a tie that the K-based version would not.

THE ONE TEST THAT CATCHES THE MOST

Zero every inactive weight, then run the ORDINARY dense matmul and compare against your masked matmul bit for bit. They must agree exactly - masking and zeroing are mathematically identical operations. This single test validates the flat-index computation, the mask plumbing, and the bit-packing convention all at once, and it is the test that catches the silent index-misalignment failure that everything else lets through.

13. Ten Defects Found While Consolidating

These are not criticisms of the source, which is a careful and unusually complete piece of documentation. They are what surfaces when fragments written in 39 separate chapters are placed side by side: inconsistencies that are invisible locally and obvious globally. Each entry gives the evidence, the consequence, and a fix.

13.1 Severity summary

ID	Defect	Severity	Fix cost
D1	DROP/GROW thresholds swap K+1 weights, not K	Low	1 line
D2	Just-dropped weights can be regrown in the same step	Medium	~6 lines
D3	GROW depends on gradients that may never be computed	Critical	Design decision
D4	Two sections give opposite call ordering	High	Doc fix
D5	Momentum buffer not cleared on regrowth	Medium	1 line
D6	AdamW memory figure counts only active weights	High	Re-plan the model
D7	Two incompatible linearConfig_t definitions	Low	Doc fix
D8	malloc of 259 KB on a 320 KB device	Critical	~15 lines
D9	T_end vs totalSteps; format version not bumped	Medium	2 lines
D10	Complexity estimate uses n where count is correct	Low	Doc fix

13.2 D1 - The thresholds are off by one

Evidence. Section 39.2's own worked example: active magnitudes [0.01, 0.05, 0.03, 0.12, 0.08] with $K = 2$ give $\text{thresh} = \text{vals}[2] = 0.05$, and the text claims 'exactly 2 weights dropped'. The condition is $|w| \leq 0.05$, which matches 0.01, 0.03 **and** 0.05 - three weights.

Consequence. Every DROP removes $K+1$ and every GROW adds $K+1$, so the active count is preserved by cancellation. The swap is one larger than requested, and where magnitudes tie - which happens constantly, because newly grown weights are all exactly 0.0 - the cancellation is no longer exact and sparsity drifts.

Fix. Make the comparison strict on one side:

```

1  /* DROP: drop exactly K */
2  if (tensorBoolGet(mask,i) && fabsf(w[i]) < dropThresh) { ... }
3
4  /* or keep <= and take the (K-1)-th value instead: */
5  float dropThresh = findAbsKthSmallestActive(weights, mask, K - 1);

```

Listing 12.1 - Two equivalent one-line fixes for D1.

13.3 D2 - Drop and grow are not disjoint

Evidence. In Listing 9.1, `findAbsKthLargestInactive()` on line 39 runs after the DROP loop has already cleared mask bits. The just-dropped weights are inactive by then, so they compete for growth.

Consequence. A weight with a small value but a steep gradient is dropped and immediately regrown - its trained value destroyed for nothing, and part of the swap budget wasted. The RigL paper draws the grow candidates from the set that was inactive **before** the drop.

Fix. Snapshot the mask before dropping, and require a grow candidate to have been inactive in the snapshot:

```

1  /* before the DROP loop */
2  tensor_t *prevMask = cloneBoolTensor(mask);
3
4  /* in the GROW loop, add one condition */

```

```

5   if (!tensorBoolGet(mask, i) &&
6       !tensorBoolGet(prevMask, i) &&          /* was ALREADY inactive */
7       fabsf(g[i]) >= growThresh) { ... }
8
9   freeBoolTensor(prevMask);

```

Listing 12.2 - Excluding just-dropped weights. Costs one extra 9 KB mask per sparse layer, transiently.

13.4 D3 - The gradients GROW needs may not exist

THIS IS THE DEFECT THAT SILENTLY TURNS RIGL INTO STATIC SPARSITY

GROW ranks inactive weights by $|\text{gradient}|$. Three separate parts of this implementation can leave those gradients at zero. (a) If the weight-gradient matmul is masked like the forward matmul (Chapter 6), inactive weights never receive a gradient at all. (b) Component 5 line 16 sets `grad[i] = 0.0f` for inactive weights on EVERY optimiser step. (c) `rigLStep()` itself zeroes them at the end of every call. If any of these wins the race, `findAbsKthLargestInactive()` sees all zeros, returns 0.0, and the GROW condition $|g| \geq 0$ matches every inactive weight - so the first K+1 encountered in index order are activated. The mask then evolves by scan order, not by gradient. Training still converges, sparsity still looks right, and RigL is not actually running.

Fix. Three things must all hold: the weight-gradient matmul stays dense; `rigLStep()` runs before the optimiser clears anything (D4); and the gradients `rigLStep()` reads are the ones from the backward pass of the same iteration. **Verify it** by asserting that `findAbsKthLargestInactive()` returns a strictly positive threshold - if it ever returns 0.0 on a layer with more inactive weights than K, you have this bug.

13.5 D4 - Contradictory ordering instructions

Section	Says	Reason given
37.4	AFTER the optimizer step, before the next forward	'This ensures that the gradient used for GROW reflects the current weight update'
39.9	BEFORE the optimizer step	'This ensures that newly grown weights ($w=0$) get their first gradient update in the same step they are activated'

Resolution. 39.9 is correct and its code implements it. Beyond the argument it gives, running after the optimiser would mean the inactive gradients had already been zeroed by Component 5 - straight into defect D3. Section 37.4 should be corrected.

13.6 D5 - Stale momentum on regrowth

Section 13.4 identifies stale Adam moments as a hazard and clears `m` and `v`. SGD with momentum has the same problem: the velocity buffer entry for a regrown weight still holds the velocity it had before it was dropped, so its first update is a jump in a stale direction. Clear it in the same branch that zeroes the weight.

13.7 D6 - The AdamW memory figure is not achievable

Evidence. Section 13.4: '7,373 active weights x 3 tensors x 4 bytes = 88 KB'. That arithmetic is right; the premise is not. `m` and `v` are allocated with the same dense shape as the parameters, so the real figure counts 73,728 elements, not 7,373: four dense float tensors plus the mask is 1,188,864 bytes for this one layer, on a device with 320 KB of SRAM.

Fix. Either store the moments sparsely - a real change, since it breaks O(1) indexing - or accept SGD, or shrink the layer. The source itself points at the third option in 39.10.

13.8 D7 - Two struct definitions

Sections 11.4 and 39.4 both present `linearConfig_t` with different field lists (Chapter 5 of this document tabulates them). Retyping either one will break the build. Edit the real header instead.

13.9 D8 - malloc on the target

`findAbsKthLargestInactive()` allocates `count * sizeof(float)` where `count` is the number of INACTIVE weights - 66,355 for the HAR layer, so 259 KB of a 320 KB device, transiently, every 100 steps, while everything else is resident. It will not allocate; and if a smaller layer lets it succeed, repeated allocation and release of large differently-sized blocks fragments the heap.

Fix. Either allocate one scratch buffer at initialisation sized for the largest sparse layer and reuse it, or replace the sort with a histogram:

```

1  /* Histogram threshold: O(n) time, O(1) memory, no allocation. */
2  #define NBINS 256
3  uint16_t hist[NBINS] = {0};
4
5  /* pass 1: find the maximum magnitude among inactive weights */
6  float gmax = 0.0f;
7  for (size_t i = 0; i < n; i++)
8      if (!tensorBoolGet(mask,i) && fabsf(g[i]) > gmax) gmax = fabsf(g[i]);
9
10 /* pass 2: bin the magnitudes */
11 float scale = (float)(NBINS - 1) / (gmax > 0.0f ? gmax : 1.0f);
12 for (size_t i = 0; i < n; i++)
13     if (!tensorBoolGet(mask,i))
14         hist[(size_t)(fabsf(g[i]) * scale)]++;
15
16 /* pass 3: walk down from the top bin until K are accounted for */
17 size_t acc = 0;
18 int b = NBINS - 1;
19 for (; b >= 0; b--) {
20     acc += hist[b];
21     if (acc >= K) break;
22 }
23 float thresh = (float)b / scale; /* approximate to one bin width */

```

Listing 12.3 - A fixed-memory replacement for the selection sort. 512 bytes of histogram instead of 259 KB of heap, and $O(n)$ instead of $O(n \cdot K)$.

IS AN APPROXIMATE THRESHOLD ACCEPTABLE?

Yes, and this is worth being explicit about. The threshold decides which connections to try next; it is re-computed 103 times over a run, and a weight that misses the cut this time will be considered again in 100 steps. Being within one bin width of the exact K-th value changes which few borderline weights are grown, not whether the algorithm works. Exactness here buys nothing and costs 259 KB.

13.10 D9 - Two smaller correctness gaps

- **T_end.** `rigLStep()` reaches $\alpha = 0$ only at `step == totalSteps`, but the schedule in section 2.3 specifies $T_{\text{end}} = 0.8 \times \text{total}$. The caller must pass `0.8f * totalSteps`, or the mask keeps swapping into the final epoch. Better: give `rigLStep()` an explicit `tEnd` parameter instead of overloading `totalSteps`.
- **Format version.** Adding the sparsity presence byte changes every tensor record, including in dense checkpoints, while chapter 17 calls the format 'locked v2'. Bump to v3 and have the reader accept both.

13.11 D10 - The complexity estimate uses the wrong n

Section 7.4 gives $n * K = 8192 \times 819 = 6.7 \text{ M}$ comparisons. The sort runs over the gathered ACTIVE values, so the cost is `count * K`. At 90% sparsity that is ten times smaller for the DROP side - and, importantly, nine times LARGER for the GROW side, where `count` is the inactive population. The corrected figures are in Chapters 3 and 4.

WHAT TO DO WITH THIS LIST

Fix D3, D8 and D4 before you run anything - they are the difference between RigL and an expensive random mask. Fix D1, D2 and D5 before you trust an accuracy number. D6 changes what model you can train at all, so settle it while the architecture is still on paper. D7, D9 and D10 are documentation and hygiene, and cost minutes.

14. Appendix - Consolidated RigL.h and RigL.c

Everything above, assembled into the two new files the implementation adds. The other components are edits to existing files and are listed in the chapters that describe them. This listing is the source's code with the D1, D2 and D9 fixes applied, and each fix marked; it is offered as a starting point, not as tested software.

14.1 RigL.h

```

1  #ifndef RIGL_H
2  #define RIGL_H
3
4  #include <stddef.h>
5  #include "Tensor.h"
6  #include "Layer.h"
7
8  /* K-selection helpers (implemented in MinMax.c) */
9  float findAbsKthSmallestActive (tensor_t *weights, tensor_t *mask,
10                                size_t K);
11  float findAbsKthLargestInactive(tensor_t *grads,   tensor_t *mask,
12                                size_t K);
13
14  /* One RigL mask update over every sparse LINEAR layer of the model.
15   * Call AFTER backward() and BEFORE the optimiser step.
16   * tEnd: the step at which alpha reaches zero (typically 0.8*total). */
17  void rigLStep(layer_t **model, size_t numLayers,
18               float alphaInit, size_t step, size_t tEnd);
19
20 #endif /* RIGL_H */

```

Listing A.1 - RigL.h.

14.2 RigL.c

```

1  #define SOURCE_FILE "RIGL"
2
3  #include "Common.h"
4  #include "RigL.h"
5  #include "Tensor.h"
6  #include "MinMax.h"
7  #include "Layer.h"
8  #include "Linear.h"
9  #include <math.h>      /* cosf, fabsf */
10
11  void rigLStep(layer_t **model, size_t numLayers,
12               float alphaInit, size_t step, size_t tEnd) {
13
14      /* --- alpha schedule; frozen once step passes tEnd (fix D9) --- */
15      if (tEnd == 0) return;
16      if (step >= tEnd) return;
17      float prog = (float)step / (float)tEnd;
18      float alpha = 0.5f * alphaInit * (1.0f + cosf(3.14159265f * prog));
19
20      for (size_t l = 0; l < numLayers; l++) {
21
22          if (model[l]->type != LINEAR) continue;
23
24          linearConfig_t *cfg = model[l]->config->linear;
25          tensor_t *mask = cfg->weightMask;
26          if (mask == NULL) continue;
27

```

```

28     tensor_t *weights = cfg->weights->param;
29     tensor_t *grads    = cfg->weights->grad;
30     size_t n = calcNumberOfElementsByTensor(weights);
31     float *w = (float *)weights->data;
32     float *g = (float *)grads->data;
33
34     size_t numActive = 0;
35     for (size_t i = 0; i < n; i++)
36         if (tensorBoolGet(mask, i)) numActive++;
37
38     size_t K = (size_t)(alpha * (float)numActive);
39     if (K == 0) continue;
40
41     PRINT_INFO("rigLStep: layer=%zu step=%zu alpha=%.4f K=%zu",
42               l, step, alpha, K);
43
44     /* snapshot of the pre-drop mask (fix D2) */
45     tensor_t *prevMask = cloneBoolTensor(mask);
46
47     /* ---- DROP: strict < drops exactly K (fix D1) ---- */
48     float dropThresh = findAbsKthSmallestActive(weights, mask, K);
49     size_t dropped = 0;
50     for (size_t i = 0; i < n; i++) {
51         if (tensorBoolGet(mask, i) && fabsf(w[i]) < dropThresh) {
52             tensorBoolSet(mask, i, false);
53             w[i] = 0.0f;
54             dropped++;
55         }
56     }
57
58     /* ---- GROW: only weights inactive BEFORE the drop (D2) ---- */
59     float growThresh = findAbsKthLargestInactive(grads, prevMask,
60                                                  dropped);
61
62     size_t grown = 0;
63     for (size_t i = 0; i < n && grown < dropped; i++) {
64         if (!tensorBoolGet(prevMask, i) &&
65             !tensorBoolGet(mask, i) &&
66             fabsf(g[i]) > growThresh) {
67             tensorBoolSet(mask, i, true);
68             w[i] = 0.0f;          /* grown connections start at 0 */
69             grown++;
70         }
71     }
72
73     if (grown != dropped)
74         PRINT_ERROR("rigLStep: layer=%zu dropped=%zu grown=%zu",
75                   l, dropped, grown);
76
77     freeBoolTensor(prevMask);
78
79     /* ---- clear gradients of everything still inactive ---- */
80     for (size_t i = 0; i < n; i++)
81         if (!tensorBoolGet(mask, i)) g[i] = 0.0f;
82 }

```

Listing A.2 - RigL.c with the D1, D2 and D9 fixes applied and marked. The **grown != dropped** check is the runtime assertion of the conservation property - if it ever fires, sparsity is drifting.

WHAT THIS LISTING IS AND IS NOT

It is a faithful consolidation of the source's algorithm with three specific defects repaired and the repairs labelled. It has not been compiled against the ODT headers, and it assumes helpers - `cloneBoolTensor`, `freeBoolTensor` - that the source does not document and you may have to write. It still uses the malloc-based K-selection, so defect D8 remains open; apply Listing 12.3 before putting this on the target.

15. Appendix - Where Every Fragment Came From

The full extraction map, so any statement in this document can be traced back to the source PDF.

Source section	Chapter of the source	Content	Used in
2.1	Part 2 intro	Why sparse training; drop/grow rationale	Ch 1
2.2	Part 2 intro	The algorithm in pseudocode	Ch 1, 9
2.3	Part 2 intro	Cosine alpha schedule and LR synergy	Ch 1, 11
2.4	Part 2 intro	Published results; the ODT gap list	Ch 1, 2
1.4	Common.h	SOURCE_FILE pattern, logging for RigL.c	Appendix A
2.4 (ch 2)	DTypes	FLOAT32 vs SYM_INT32 access	Ch 3, 9
7.4	MinMax	Both K-selection functions, complexity	Ch 3, 4, 12
8.4	Matmul	Mask-aware inner loop; counter verification	Ch 6
9.4	RNG/Bernoulli	Mask initialisation and seeding	Ch 11
11.4	Linear	weightMask field; mask memory	Ch 5
12.4	Sgd	Mask-aware SGD kernel	Ch 7
13.4	AdamW	Mask-aware AdamW; moment clearing	Ch 8, 12
14.4	LrScheduler	LR and alpha schedule together	Ch 11
17.5	Serialize	serializeSparsity, deserializeSparsity	Ch 10
20.4	Conv1d	kernelMask extension (not implemented)	Ch 2
33.4	Comparison	DROP/GROW as threshold comparisons	Ch 1
37.4	TrainingLoopApi	Integration point; ordering claim	Ch 11, 12
38.4	UserAPI	Mask creation in buildSequentialModel	Ch 11
39.1-39.10	RigL Complete	All seven components; training loop; expected results	Ch 3-11

Sections 3.4, 4.4, 5.4, 6.4, 10.4, 15.6, 16.4, 18.4, 19.4, 21.4, 22.4, 23.4, 24.4, 25.5, 26.4, 27.4, 28.4, 29.6, 30.4, 31.4, 32.5, 34.5, 35.4 and 36.4 were read and contain no RigL code - they state that the file in question needs no change, which is itself useful information: it means the mask does not have to be threaded through activations, pooling, normalisation, flattening, the data loader, or the PPCA replay path.